Ticket #004 - Threat Hunt Report

Executive Summary

This hunt began after unusual activity was reported on an Ubuntu Linux lab host, but there was no high-confidence alert, confirmed incident, or known IOC to work from. I started with a hypothesis that unauthorized persistence might be occurring through scheduled execution mechanisms such as cron or systemd.

The initial cron review identified two recent jobs. `dev-health-check` had a straightforward monitoring purpose. `config-backup` deserved more attention because it ran every five minutes as root, collected host and SSH configuration, and wrote compressed archives to `/var/tmp`.

The investigation changed direction once the surrounding context was correlated. Sudo records showed that the `analyst` account created both jobs during an interactive session. SSH telemetry tied that session to `192.168.56.1`, and historical login records showed repeated prior analyst sessions from the same source.

A separate hunt pivot uncovered `/usr/local/bin/detect-ssh-bruteforce.sh` running every minute from the root crontab. Script inspection showed it was monitoring failed SSH logins and generating warnings, which was consistent with a defensive control. The systemd review did not identify another unexplained persistence mechanism.

Final disposition: Close - Benign

Confidence: Medium

The available evidence is most consistent with legitimate lab and administrative activity. Confidence remains Medium because no independent change-management record was available to prove that the changes were formally approved.

Hunt Hypothesis

If unauthorized persistence had been established through Linux scheduling mechanisms, I expected to find recent or unexplained cron or systemd changes that launched unusual commands, involved unexpected users or paths, or ran on a pattern that did not fit normal administration.

The hunt was designed to test that idea, not to confirm it. Benign explanations were kept in play throughout the investigation.

Starting Point

Known facts

At the beginning of the hunt:

- An analyst had noticed activity that seemed unusual.
- No high-confidence security alert had fired.
- No incident had been confirmed.

- There was no starting IOC.
- The target was an authorized Ubuntu Linux lab host.
- The SOC Manager had requested a proactive, hypothesis-driven hunt.

Unknowns

The investigation still needed to establish:

- What activity caused concern.
- Which account performed it.
- Whether the activity was authorized.
- Whether cron, systemd, or another persistence method was involved.
- Whether privilege escalation occurred.
- Whether the activity came from a local or remote session.
- Whether suspicious scripts or processes were involved.
- Whether the behavior extended beyond one host.

Assumptions

I assumed that relevant logs and configuration data were still available, that legitimate scheduled administration was possible, and that recent scheduling changes should leave observable traces in configuration files, metadata, authentication records, privilege logs, or execution telemetry.

I also treated "unusual" and "malicious" as different things. A new root-level job could be a useful lead without being an incident.

Competing Explanations

I kept four broad explanations in mind:

1. **Routine administration:** monitoring, backups, maintenance, or automation.
2. **Legitimate but poorly documented change:** technically valid activity without a formal change record.
3. **Unauthorized persistence:** a scheduled mechanism created to maintain continued execution.
4. **Application or package automation:** a service or installed component creating scheduled activity automatically.

Maintaining benign alternatives from the start helped reduce confirmation bias.

Hunt Plan

The initial plan covered:

- Cron configuration

- User crontabs
- Systemd services and timers
- Authentication telemetry
- Sudo and privileged activity
- File metadata
- Script contents
- Journal and execution evidence
- Historical login behavior

The scope would expand only when a finding created a specific new question.

Environment and Data Sources

System

- **Host:** `ubuntu-soc-lab`
- **Operating system:** Ubuntu Linux
- **Primary investigative account:** `analyst`

Cron and crontab data

Used to identify recent scheduled jobs, their users, commands, and execution frequencies.

Systemd services and timers

Used to look for another persistence or recurring-execution path outside cron.

Authentication logs

Used to identify the source and timing of interactive access.

Sudo telemetry

Used to determine which account performed privileged configuration changes.

File metadata and artifacts

Used to correlate timestamps, ownership, permissions, and archive creation with the scheduled tasks.

Journal and login history

Used to confirm actual scheduled execution and compare the current session with prior analyst behavior.

Investigation

Cron Enumeration

The cron review identified two recent jobs.

`dev-health-check`

- Runs every 15 minutes as root.
- Executes `/usr/local/bin/dev-health-check.sh`.
- Records hostname, uptime, and filesystem usage.
- Writes to `/var/log/dev-health-check.log`.

The task had a clear monitoring purpose and did not raise significant concern after inspection.

`config-backup`

- Runs every five minutes as root.
- Executes `/usr/local/bin/config-backup.sh`.
- Archives `/etc/hosts` and `/etc/ssh/sshd_config`.
- Writes timestamped archives under `/var/tmp`.

This job warranted a deeper look because it combined frequent root execution, SSH configuration collection, and temporary-directory storage.

Script and Artifact Review

The backup script created files such as:

`/var/tmp/dev-config-20260823-175433.tar.gz`

The observed archives were root-owned, mode 600, approximately 1.9 KB, and stored under `/var/tmp`.

Multiple files confirmed repeated execution. Journal records independently showed cron running the backup around 18:00:01 UTC and again at 18:05:01 UTC.

That was important because the hunt was dealing with active recurring behavior, not merely a dormant configuration file.

Authentication and Privilege Correlation

Sudo telemetry showed that the `analyst` account:

- Created `/usr/local/bin/dev-health-check.sh`.
- Made it executable.

- Created `/etc/cron.d/dev-health-check`.
- Created `/usr/local/bin/config-backup.sh`.
- Made it executable.
- Manually ran `config-backup.sh`.
- Created `/etc/cron.d/config-backup`.
- Set permissions on the cron files.

One sudo authentication failure occurred at 17:50:04 UTC, followed about five seconds later by successful sudo activity from the same account and session. In context, the isolated failure had little investigative weight.

The larger finding was attribution: the recent jobs were tied to a visible interactive analyst session rather than an unexplained background process.

Session Origin and Historical Context

The active analyst `pts/0` session originated from `192.168.56.1`. SSH records showed successful authentication at 17:35:59 UTC, immediately before the scheduled-task changes.

Historical login records showed repeated prior analyst sessions from the same source address.

That pattern supported a benign explanation, but it was not treated as proof of authorization. A familiar source can still be compromised, and the hunt did not have an authoritative change or identity record to independently verify approval.

Hunt Pivot

What appeared

During cron execution review, another script was observed running every minute:

```
/usr/local/bin/detect-ssh-bruteforce.sh
```

It had not appeared in the initial `/etc/cron.d` enumeration.

Why it mattered

The job ran as root every minute, so it represented another persistent scheduled execution mechanism. The security-related filename was not enough reason to trust it without inspection.

What the follow-up showed

The root user's crontab contained:

```
* /usr/local/bin/detect-ssh-bruteforce.sh
```

The script:

- Reviews recent SSH journal events.
- Searches for failed password attempts.
- Counts failures by source IP.
- Uses a threshold of five failures within five minutes.
- Generates an `auth.warning` message when the threshold is reached.

Its timestamp was July 20, 2026, predating the current scenario.

Pivot assessment

The pivot weakened the original hypothesis. The job's behavior was consistent with defensive SSH brute-force monitoring rather than unauthorized persistence.

This was one of the clearest examples in the hunt of why scheduling frequency and root privileges are context, not a verdict.

Systemd Review

The timer review showed normal Ubuntu maintenance activity, including `apt-daily`, `apt-daily-upgrade`, `logrotate`, `fstrim`, `sysstat`, `dpkg-db-backup`, `fwupd-refresh`, and `systemd-tmpfiles-clean`.

One custom service stood out:

`/etc/systemd/system/company-web.service`

The service predated the hunt, ran as `analyst`, launched `/usr/bin/python3 /home/analyst/internal-web-outage-lab/app.py`, and served the application locally at `127.0.0.1:5050`.

The available evidence was consistent with an existing internal Flask lab application. No additional unexplained systemd persistence was identified.

Evidence Classification

Facts

- The analyst account authenticated over SSH from `192.168.56.1`.
- The analyst account used sudo to create both recent cron tasks.
- `config-backup.sh` archives `/etc/hosts` and `/etc/ssh/sshd_config`.
- Backup archives were written under `/var/tmp`.
- Cron executed the backup automatically.
- The root crontab runs `detect-ssh-bruteforce.sh` every minute.
- That script checks failed SSH authentication attempts.
- Historical analyst logins repeatedly originated from `192.168.56.1`.

- No new suspicious systemd timer was identified.

Inferences

- `dev-health-check` is consistent with legitimate monitoring.
- `config-backup` is most consistent with administrative or lab activity.
- `detect-ssh-bruteforce` is likely a defensive control.
- `company-web.service` is consistent with a legitimate internal lab application.
- The combined evidence does not currently justify incident escalation.

Assumptions

- The analyst activity was formally authorized.
- Historical source consistency reflects expected ownership of the session.
- The company web application was formally approved.

Those assumptions could not be independently verified through change-management records.

Findings

The backup job initially presented several characteristics that justified investigation:

- Root execution
- A five-minute schedule
- SSH configuration collection
- Storage under `/var/tmp`

The meaning of those signals changed once they were correlated with account and session evidence. The task was created by the analyst account during an interactive SSH session whose source matched historical analyst activity.

The separate root-cron pivot also ended with a benign explanation, and the systemd review did not reveal another unexplained persistence mechanism.

The hunt found no evidence of:

- Account compromise
- Unauthorized login
- Malware
- External command and control
- Data exfiltration
- Credential theft
- Hidden malicious systemd persistence

- Unauthorized user creation
- Security-control disabling

Disposition

Close - Benign

The activity was unusual enough to investigate, but the evidence across authentication, sudo, scheduled execution, file artifacts, script contents, and historical login behavior provided a coherent benign explanation.

Escalation is not recommended at this time.

Confidence Assessment

Confidence: Medium

The conclusion is supported by multiple independent telemetry sources. The strongest evidence was the correlation between:

- The analyst SSH session
- Historical use of `192.168.56.1`
- The sudo commands that created the jobs
- The actual script contents
- Cron execution telemetry

Missing evidence

The primary gap is independent confirmation that the changes were formally approved. There was no change ticket, separate administrator approval record, centralized identity source, or authoritative asset record available.

What could raise confidence

Confidence could increase to High if:

- A change-management record confirmed the scheduled-task creation.
- An administrator independently confirmed the work.
- Asset records identified `192.168.56.1` as the expected administrative source.
- Centralized identity telemetry independently confirmed the session.

What could change the conclusion

The assessment would need to be revisited if new evidence showed:

- Account compromise

- An unexpected login source
- Stolen credentials
- Hidden persistence
- External communication tied to the scheduled job
- Unauthorized access to the archives
- Malicious modification of the scheduled scripts

Limitations

This hunt took place in a controlled lab environment. The main limitations were:

- No centralized SIEM
- No enterprise EDR telemetry
- No formal change-management database
- No identity-provider telemetry
- No authoritative asset ownership records
- Limited historical process telemetry
- No enterprise-wide search across other hosts

These gaps mostly affected the ability to independently confirm formal authorization.

Recommendations

1. Centralize Linux authentication, sudo, cron, and systemd logs.
2. Use file-integrity monitoring for `/etc/cron*`, root crontabs, `/etc/systemd/system`, and `/usr/local/bin`.
3. Maintain an approved baseline of scheduled tasks.
4. Alert on new root-level cron jobs and systemd services.
5. Correlate administrative changes with formal change-management records.
6. Monitor scheduled scripts that execute from or write to temporary directories.
7. Retain process execution telemetry so parent-child relationships can be reconstructed.
8. Document defensive automation such as `detect-ssh-bruteforce.sh` so future analysts can identify its purpose quickly.
9. Review root user crontabs in addition to `/etc/cron.d`.
10. Avoid classifying persistence mechanisms as malicious without account, command, file, network, and historical context.

Stopping Criteria

The hunt stopped after the original hypothesis and major competing explanations had been tested across cron configuration, cron execution, script contents, file artifacts, sudo activity, SSH authentication, session origin, historical

access, the root-crontab pivot, and systemd.

At that point, there was no remaining evidence-driven lead that was likely to change the disposition. More collection was possible, but continuing without a specific unanswered question would not have improved the decision.

Final Assessment

Disposition: Close - Benign

Confidence: Medium

The available evidence does not support escalation to formal incident response. The hunt showed that root execution, frequent schedules, temporary storage, and persistent execution can all be useful hunting signals, but they only become meaningful when interpreted alongside identity, privilege, script behavior, execution, and historical context.